PCAP Network Analysis

Anuj Kshatriya
23110173

Ayush Sabane
23110054

Ankit Kumar
23110030

Sanskar Prajapati
23110290

Ashutosh Sahu
23110050

Raj Kumar Gupta
23110267

I. Introduction

Packet captures (PCAP files) show exactly what happened on a network. Tools like Wireshark decode them well, but understanding a capture still means scrolling through thousands of packets and knowing what each label really proves. For example, a retransmission does not always mean congestion.

This project is an **offline PCAP analysis tool**. A Python **engine** lets TShark decode the packets, then groups and calculates the results. A **FastAPI** backend serves the engine, a **React** dashboard shows an overview first and details on click, and two **Jupyter notebooks** use the same engine.

Contributions. (1) One TShark-based engine shared by the API and the notebooks. (2) Detailed TCP analysis: flows, RTT, flow control, options and a careful congestion check. (3) Detail pages for UDP, DNS, TLS, QUIC, ARP and IP; all other protocols are counted on the overview.

II. Problem Statement

Problem. Given a PCAP or PCAPNG file, show which protocols are present, who talks to whom, how each TCP connection behaves, and what the application protocols did, without presenting guesses as facts.

Input. One capture file, uploaded in the browser or passed to the engine from a notebook. **Output.** An overview page, detail pages (metrics, charts, tables, packet decode) and a JSON report with warnings.

Assumptions and limits. Analysis is offline and the capture comes from one point in the network. TShark 4.x must be installed. No decryption keys are used, so TLS and QUIC data stay hidden. Results are kept in memory (about 4 KB per packet).

III. Project Objectives

- **O1 Input.** Accept PCAP/PCAPNG; reject bad files with a clear message.
- **O2 Overview first.** Show capture facts and all protocols; detail pages only for TCP, UDP, DNS, TLS, QUIC, ARP and IP.
- **O3 TCP depth.** Per-flow packets, bytes, throughput, retransmissions, out-of-order, duplicate ACKs, RTT, flow control, options and congestion hints.
- **O4 Scale.** Analyse at least 250,000 packets in under one minute on a laptop.

V. Architecture and Design

Fig. 1 shows the system. The browser sends JSON requests to a thin FastAPI layer (upload, analyse, report, packet pages, packet decode). All analysis is in the **engine**. One TShark runner builds every command. A chunked extractor turns TShark output into a table, and the analysers turn it into page descriptions (metrics, charts, tables) that the dashboard and notebooks draw.

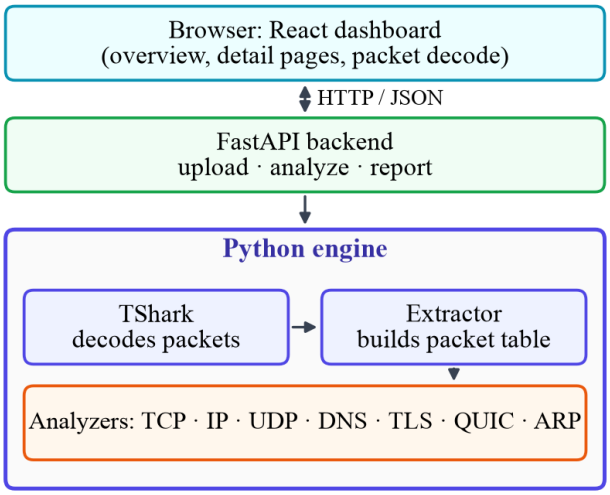


Fig. 1. System architecture.

A. Components

- **Upload.** Checks the file header and gives readable errors.
- **Engine.** Whole-capture totals from TShark statistics, plus a detailed pass over the packets.
- **Analyzers.** TCP, IP, UDP, DNS, TLS, ICMP, HTTP, DHCP, RTP and a generic view.
- **Dashboard.** Overview, protocol cards, top navbar and tabs; any packet row opens its full decode.

B. Design decisions

- **TShark for decoding.** Wireshark's analysis is the reference users trust, so the engine uses its flags.
- **Page specs as data.** One generic renderer serves many protocols; layouts are less custom.
- **Results in memory.** Simple and fast, but lost on restart.
- **Observed/derived/inferred.** Each value shows how it was obtained.

C. Request flow

The user uploads a file; the backend saves and checks it. **Analyse** then runs: (1) whole-capture totals; (2) packet extraction; (3) TCP analysis; (4) protocol detection and analysers, in parallel; (5) summary and warnings. The browser fetches only what a page needs, and a packet decode runs tshark -V -x for that one packet.

D. Dashboard

The first page shows capture facts, a protocol tree, a chart of all protocols, cards for protocols with detail pages, and a table for the other protocols. A navbar is on every page. Detail pages use tabs; tables can be sorted, searched and paged.

VI. Implementation

Stack. Python 3.14 (FastAPI, pandas, NumPy), TShark 4.6.9, React 19, Vite and ECharts, and Jupyter. Table I lists the engine modules.

TABLE I. ENGINE MODULES

Module	Job
tshark	Builds and runs TShark commands; checks which fields exist
capture	Validation and whole-capture statistics
extract	Chunked, typed packet extraction
tcp, tcp_views	Flows, RTT, window scaling, options, congestion; per-flow views
ip, udp, dns, tls, icmp, http, dhcp, rtp, generic	Protocol analyzers
detect	Protocol detection from TShark's protocol chain
charts, util, config, errors	Helpers and error messages

A. TCP analysis

Flows follow tcp.stream. The client is the sender of the first pure SYN. The flow table has over 35 columns, including packets, bytes, duration, throughput, retransmissions, out-of-order, duplicate ACKs, zero windows and RTT. The dashboard shows ACK-based RTT and handshake RTT. Flow control is tracked per direction, and TCP options (MSS, window scale, SACK, timestamps, ECN) are split into those set in the handshake and those seen later.

B. Algorithms

Algorithm 1: effective receive window. Window scaling applies only if both SYNs carry the option.

```
client := sender of first pure SYN (else first packet)
shift[s] := window-scale option in side s's SYN
scaled := both SYNs captured and both have option
for each packet p from side s:
    if p is SYN: eff := p.window
    elif both SYNs captured:
        eff := p.window * 2^shift[s] if scaled else p.window
    else: eff := p.window; status := uncertain
```

Algorithm 2: congestion check (inferred). A flow is flagged only when several signs appear together.

```
loss := retx>0 or lostSeg>0 or dupACK>=3
rtt  := p95/min >= 2 and samples >= 10
ecn  := CE>0 or (ECN negotiated and ECE>0)
if few data packets and no loss/ecn/rwnd: insufficient
elif ecn: possible congestion (ECN signal)
elif loss and rtt: possible congestion
elif loss: possible loss or reordering
elif reorder/rwnd/rtt: weaker label
```

Conversation key. Conversations are grouped by (protocol, sorted endpoints), so two conversations with the same IPs but different ports are never merged.

C. Challenges and solutions

- **Field names differ between TShark versions.** The runner reads tshark -G fields and reports missing fields as unavailable.

Repository. https://github.com/ankiit-18/PCAP-Analyzer-CN-Project

Folder layout. code/ holds backend/ (main.py, engine/), frontend/, notebook/, samples/ and start.sh/start.bat. ai/ holds the prompts, report/ this report, ppt/ the slides.

VII. Testbed Setup

Hardware and OS. Apple MacBook Air, M4 chip, 16 GB RAM, macOS 15.7.4. No GPU or sensors. **Software.** Python 3.14.0, TShark 4.6.9, FastAPI 0.162.0, pandas 3.1.0, NumPy 2.5.4, pytest 9.2.0, Node 24.1.0, npm 11.3.0, React 19, Vite 8, ECharts 6.

Topology. No live network. Captures are analysed offline on one machine; the browser connects to 127.0.0.1:8000.

VIII. Evaluation

A. Dashboard overview

We ran the dashboard on a real capture (8,321 packets, 5.5 MB, 41.6 s). The overview page in Fig. 2 is the first thing the user sees. The top shows the capture facts: duration, packets,

bytes, packet rate, throughput, protocols, endpoints and conversations. Below are highlights (for example, TCP retransmissions and DNS queries with no response), a protocol tree and a chart of packets per protocol.

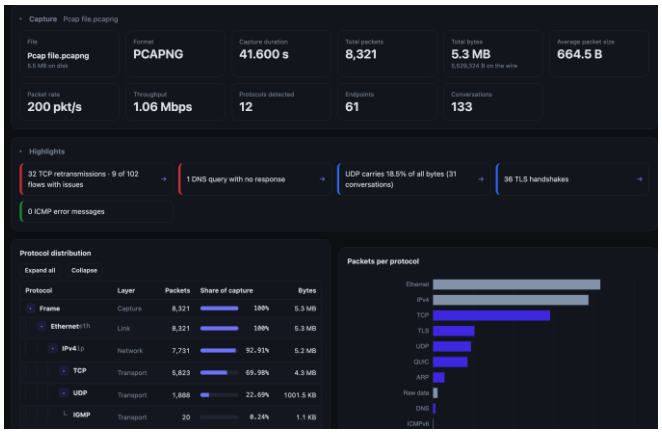


Fig. 2. Overview page.

Clicking a card or the top navbar opens a detail page. Fig. 3 shows the TCP page: counts of retransmissions, duplicate ACKs, out-of-order packets and sequence gaps, the average RTT, and the flows with the most issues.

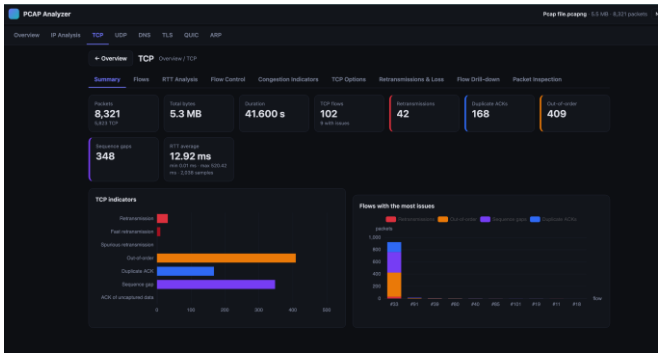


Fig. 3. TCP summary page.

Bad input

Table VI shows how the system handles bad or unusual input.

TABLE VI. INPUT HANDLING

Case	Behaviour
Empty or tiny file	Rejected with a message
Not a PCAP/PCAPNG file	Rejected with export advice
Corrupt file body	Readable engine error
TShark not installed	Message with install hints
Capture without TCP	Runs, with a warning
Handshake not captured	Window scale "uncertain"

IX. Summary

We built an offline PCAP analysis tool. The engine lets TShark decode packets and adds clear summaries and careful inference. The dashboard shows an overview first, with detail pages for TCP, UDP, DNS, TLS, QUIC, ARP and IP. **Limitations.** Memory grows with packet count; results are not saved; Analyse is a blocking request. **Future work.** Background jobs with progress, a database for very large captures, QUIC-specific analysis, tests on large real captures.

IX. Reproducibility

Repository: https://github.com/ankiit-18/PCAP-Analyzer-CN-Project.

Setup. Install TShark (brew install wireshark or sudo apt install tshark), Python 3.10+ and Node 18+. From the code folder:

```
./start.sh            # builds UI, serves :8000
# open http://127.0.0.1:8000, upload a sample
cd backend && pip install -r requirements-dev.txt
```

References

[1] Anthropic, “Claude Sonnet 5.5,” large language model, Anthropic, San Francisco, CA, USA, 2026. [Online]. Available: https://www.anthropic.com/claude

APPENDIX A: TEAM WORK DISTRIBUTION AND MEMBER RATING

A.1 Work Distribution

- *Ratings are agreed by the whole team and must be UNIQUE; no two members may receive the same rating (no ties).*
- *Scale: 1–10 (10 = highest contribution). With 3 members, use three different values (e.g., 10, 9, 8).*

#	Member Name	Roll No.	Tasks Done	Rating (1–10, unique)
1	Anuj Kshatriya	23110173	Python Backend, Frontend & PCAP Analysis Engine. Building the core functionality of the project.	10
2	Ayush Sabane	23110054	Python Backend, Frontend & PCAP Analysis Engine. Building the core functionality of the project.	9
3	Sanskar Prajapati	23110290	Final report and ppt making, testing current dashboard result with wireshark analysis.	8
4	Ankit Kumar	23110030	Wireshark analysis, TCP analysis features building.	7
5	Ashutosh Sahu	23110050	Final report, ppt making and final compilation, frontend design changes.	6
6	Raj Kumar Gupta	23110267	UI template searching for dashboard.	5

APPENDIX B: AI USAGE DETAILS

B.1 Prompts

#	AI Tool	Purpose
1	Claude (Sonnet 5.5)	Turning the project brief into the analysis notebook based on TShark commands. Writing the Python engine, FastAPI backend, React/ECharts dashboard and notebooks and debugging.
2	Claude (Sonnet 5.5)	Dashboard changes requested by the team and drafting this report.
3	ChatGPT (https://chatgpt.com/share/6aecb6d9-2000-83e8-982a-97de6d3b1b54) (https://chatgpt.com/share/6aecc554-e9e0-83e8-897b-5b43a475a5f4?ogimg=plain)	We used ChatGPT for project planning, code development and debugging, PCAP analysis improvements, and final report preparation. Also used for understanding project, Wireshark.